

HTML5: tag semantici

Confronto con <div> + id/class e buone pratiche moderne

Obiettivo: markup più leggibile, accessibile e manutenibile

Il problema: “div soup”

Struttura comprensibile per l'umano, ambigua per le macchine

Con `<div class="header">` / `<div class="main">` la semantica è implicita e dipende dalle convenzioni di naming.

```
1 <div class="header">
2   <h1>My Site</h1>
3   <div class="nav">...</div>
4 </div>
5
6 <div class="sidebar">...</div>
7 <div class="main">...</div>
8 <div class="footer">...</div>
```

Elemento “semantico”

Non “magia grafica”: è significato standardizzato

- Un tag semantico descrive il ruolo del contenuto (non lo stile).
- Di default si comporta come un blocco: styling via CSS.
- Benefici: lettura da parte di screen reader, crawler, strumenti di analisi.
- Riduce ambiguità rispetto a class/id arbitrari.

Idea chiave

Un layout fatto con `<header>/<nav>/<main>/<footer>` è “interpretabile” anche senza conoscere le classi CSS associate.

I principali “landmark” HTML5

Strutturano la pagina e aiutano la navigazione assistiva

`<header>`

`<nav>` (solo navigazioni “principali”)

`<main>`

`<section>` / `<article>` ...

`<aside>`
contenuto
complementare

`<footer>`

Esempio: pagina “blog post”

Confronto pratico tra HTML “classico” e HTML5 semantico

Prima: <div> + class

```
1 <div class="header">...
2   <div class="nav">...</div>
3 </div>
4 <div class="sidebar">...</div>
5 <div class="main">...
6   <div class="post">...</div>
7 </div>
8 <div class="footer">...</div>
```

Dopo: tag semantici

```
1 <header>...
2   <nav>...</nav>
3 </header>
4
5 <main>
6   <article>
7     <header>...</header>
8     <section>...</section>
9   </article>
10  <aside>...</aside>
11 </main>
12
13 <footer>...</footer>
```

Mappatura tipica

Come “tradurre” classi/ID in struttura semantica

Prima (div + class/id)	Dopo (tag semantici)
<code>div.header / #header</code>	<code><header></code> (può contenere brand + navigazione)
<code>div.nav / ul.menu</code>	<code><nav></code> (solo blocchi di navigazione importanti)
<code>div.main / #content</code>	<code><main></code> (contenuto principale unico per pagina)
<code>div.post / .entry</code>	<code><article></code> (unità pubblicabile: post, commento, news)
<code>div.section</code>	<code><section></code> (sezione tematica; idealmente con heading)
<code>div.sidebar</code>	<code><aside></code> (contenuti complementari: ads, TOC, related)
<code>div.footer</code>	<code><footer></code> (metadati, copyright, link correlati)

<section> vs <div>

Regola pratica: section ha “tema”, div è neutro

- Usa <div> per raggruppamenti puramente “tecnici” (grid, wrapper, layout).
- Usa <section> per una sezione tematica del contenuto.
- Buona prassi: una <section> dovrebbe avere un heading (h2/h3...).
- Non usare <section> come sostituto universale di <div>.

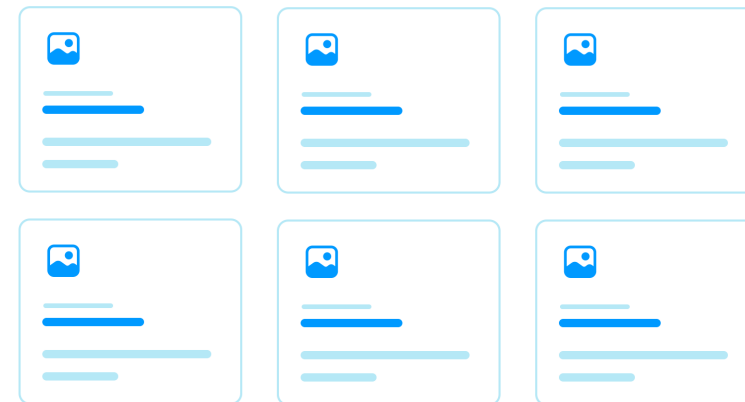
Esempio: “Recensioni”, “FAQ”, “Dettagli prodotto” sono sezioni.
“container”, “row”, “col-6” sono wrapper: meglio <div>.

<article>

Unità autocontenuta e riutilizzabile

- Contenuto “pubblicabile” da solo: post, news, commento, scheda prodotto.
- Può contenere un suo <header>/<footer>.
- Può essere annidato (es. commenti come article dentro article).
- Spesso è il nodo che finisce in un “card” feed.

```
1 <article>
2   <header>
3     <h2>Titolo</h2>
4     <p><time datetime="2026-03-02">2 marzo 2026</time></p>
5   </header>
6   <p>...</p>
7   <footer>Tag, autore, link correlati...</footer>
8 </article>
```



<nav>

Solo “maior navigation blocks”

- Esempi: menu principale, indice (TOC), breadcrumbs importanti.
- Non ogni lista di link deve essere <nav>.
- Se hai più nav, differenziale con aria-label (es. “Primaria”, “Secondaria”).

```
1 <nav aria-label="Navigazione principale">
2   <ul>
3     <li><a href="/">Home</a></li>
4     <li><a href="/docs">Docs</a></li>
5   </ul>
6 </nav>
```

<header> e <footer>

Non solo “testata/piede pagina”

- Possono comparire a livello di pagina o all’interno di <article>/<section>.
- <header>: titolo, intro, meta, strumenti contestuali.
- <footer>: meta, link correlati, autore, licenza, “fine sezione”.

```
1 <article>
2   <header>
3     <h2>FAQ</h2>
4     <p class="lede">Domande frequenti...</p>
5   </header>
6   ...
7   <footer>
8     <a href="#top">Torna su</a>
9   </footer>
10 </article>
```

<aside>

Contenuto complementare (non principale)

- Sidebar, related content, glossario, annunci, box informativi.
- Può essere globale (layout) o “a bordo” di un articolo.
- Se è contenuto principale, NON è aside.

```
1 <main>
2   <article>...</article>
3   <aside>
4     <h2>Articoli correlati</h2>
5     <ul>...</ul>
6   </aside>
7 </main>
```

Due tag “piccoli” ma utili

<time> e <mark>

<time>

Rende machine-readable date/ora (datetime).

```
1 <time datetime="2012-05-01T12:30+00:00">  
2   01 Mag 2012, 12:30  
3 </time>
```

<mark>

Evidenzia un frammento rilevante (es. highlight risultati).

```
1 <p>Risultati per:  
  <mark>semantica</mark></p>
```

Nota: <mark> è semantico (non “solo giallo”).
Lo stile è CSS.

<main>

Unico per pagina, racchiude il contenuto principale

- Tipicamente 1 solo <main> per documento.
- Esclude header, nav, footer, sidebars ripetitive.
- Migliora “skip navigation” e landmark per screen reader.

```
1 <a class="skip" href="#main">Salta al contenuto</a>
2 ...
3 <main id="main">
4   ...
5 </main>
```

Classi/ID: non spariscono

Semantica = struttura: classi/ID = styling e behavior

- Usa classi per tutto ciò che riguarda l'aspetto visivo (BEM/utility); per il JavaScript meglio usare gli attributi data-* (es. data-dropdown) o selettori specifici per i test, per evitare che il codice JS si rompa in caso di cambiamento di nome di una classe CSS.
- Usa id per unicità, es. ancore, collegamento di label ai loro input (con for), target di ARIA ...
- Evita di “codificare” semantica solo nel naming (main/body/content...).
- Preferisci: semantica nel tag + dettagli nel nome classe.

```
1 <header class="site-header">
2   <nav class="site-nav" aria-label="Primaria">...</nav>
3 </header>
4
5 <main class="layout" id="main">...</main>
```

BEM (Block Element Modifier)

Metodologia di naming di classi CSS

- Definizione: È una metodologia di naming per le classi CSS nata in Yandex.
- Filosofia: Trattare l'interfaccia come un insieme di componenti indipendenti.
- Obiettivo: Evitare conflitti tra stili e rendere il codice "auto-esplicativo".
- Best Practice: Si sposa perfettamente con il principio di usare le classi per definire lo stile e i tag per la semantica.

Struttura BEM (Block Element Modifier)

Anatomia di una classe BEM

- La sintassi segue lo schema: blocco__elemento--modificatore
- Block (Blocco): L'entità astratta o componente principale (es. .menu, .card). È riutilizzabile e indipendente.
- Element (Elemento): Una parte del blocco che non ha un significato autonomo (es. .card__title, .menu__item). Si indica con due underscore (__).
- Modifier (Modificatore): Una variante o un cambio di stato (es. .card--featured, .menu__item-active). Si indica con due trattini (--).

```
.header__nav--primary
```


Esempio BEM (Block Element Modifier)

Esempio pratico e confronto

- Vantaggi:
 - Bassa Specificità: Si usano quasi sempre selettori a classe singola, evitando la "guerra dei pesi" nel CSS.
 - Modularità: Puoi spostare il blocco card ovunque senza che lo stile si rompa.
 - Chiarezza: Il team sa esattamente dove mettere le mani senza aver paura di rompere altri componenti.

```
<article class="card card--highlight">  
  <h2 class="card__title">Titolo della Card</h2>  
  <p class="card__description">Contenuto descrittivo...</p>  
  <button class="card__button card__button--large">Leggi  
tutto</button>  
</article>
```

- Il Problema: Se usi una classe come `.btn-primary` per attivare una funzione JS, il codice diventa fragile.
- Conseguenza: Un grafico cambia il nome della classe per scopi estetici (es. in `.btn-large`) e involontariamente "rompe" le funzionalità JavaScript della pagina.
- La Soluzione: Usare classi esclusivamente per il CSS (seguendo BEM o utility) e attributi `data-*` o selettori di test per JavaScript.
- Vantaggio: Puoi stravolgere il design del sito senza mai dover toccare una riga di codice JavaScript.
- L'uso degli attributi `data-*` come "agganci" (hook) per JavaScript è uno dei pilastri della manutenibilità nel frontend moderno. L'idea centrale è il disaccoppiamento: separare ciò che definisce l'aspetto (CSS) da ciò che definisce il comportamento (JS).

Attributi data-*

HTML5 e gli attributi personalizzati

- Gli attributi data-* sono considerati attributi globali in HTML5. Puoi aggiungerli a un <div>, a un , a un <main> o a qualsiasi altro tag semantico.
- Cos'è data-js? È un nome arbitrario scelto per identificare gli elementi che hanno bisogno di JavaScript, distinguendoli da quelli che hanno solo bisogno di CSS.
- Quali altri esistono?: In realtà, non esiste una lista predefinita. Il bello di HTML5 è che dopo il prefisso data- puoi scrivere quello che vuoi!
- Regole di Naming:
 - Deve iniziare con data-.
 - Deve contenere solo minuscole (nel codice HTML).
 - Non deve contenere spazi.

Attributi data-*

data-is il gancio per JavaScript

- data-js è un attributo personalizzato (Custom Data Attribute) creato dallo sviluppatore.
- Scopo: Serve a "marcare" un elemento HTML come interattivo, dicendo al programmatore: "Ehi, questo elemento è controllato da uno script!".
- Perché si usa?: Per separare nettamente le responsabilità.
- Le classi servono per il CSS (estetica).
- I data-attributes (come data-js) servono per il JavaScript (comportamento).
- Vantaggio: Se un domani decidi di cambiare il nome della classe CSS per scopi di refactoring, il tuo codice JavaScript non smetterà di funzionare perché "agganciato" al data-js e non alla classe.

Classi BEM e Attributi data-*

Un esempio

- In questo esempio:
 - Se cambi la classe c-button--large, il JS che punta a data-js continua a funzionare.

Categoria	Attributo nel codice	Regola della Slide
Estetica (CSS)	class="c-button..."	Usa classi per CSS (BEM)
Logica (JS)	data-js="..."	Usa data-* per agganci JS
Accessibilità	aria-controls="..."	Usa target di ARIA
Identità Unica	id="submit-form-1"	Usa id per hook unici/ancore
Automazione	data-testid="..."	Usa test selectors

```
<button
  type="button"
  id="submit-form-1"
  class="c-button c-button--large"
  data-js="validate-and-send"
  data-testid="submit-btn"
  aria-controls="feedback-msg">
  Invia Richiesta
</button>
```

ARIA e semantica

Prima usa HTML: poi ARIA (se serve)

- Molti tag hanno ruoli impliciti (landmark). Evita ruoli ridondanti.
- Usa ARIA per: etichettare nav multiple, widget custom (combobox, tab, menu...).
- Non cambiare il ruolo nativo in modo incoerente (rischio accessibilità).

```
1 <!-- Nav multiple: etichetta -->
2 <nav aria-label="Secondaria">...</nav>
3
4 <!-- Widget custom: serve ARIA -->
5 <input role="combobox" aria-autocomplete="list" ...>
```

Benefici pratici

Accessibilità, SEO, manutenzione, tooling

- Accessibilità: landmark e struttura di heading più navigabili.
- SEO: parsing più robusto (titoli, article, time), rich results con dati strutturati.
- Manutenzione: leggibilità del markup e refactor più sicuri.
- Tooling: audit più chiari (Lighthouse/axe) e test E2E più stabili.

Regola d'oro: "HTML semantico prima, CSS/JS dopo".

Compatibilità e fallback

In generale: ok. Eccezione storica: IE≤8

- Browser moderni renderizzano elementi sconosciuti come inline e li puoi “bloccare” via CSS.
- IE≤8 non permetteva styling su tag sconosciuti → serviva “HTML5 shiv”.
- Oggi: quasi sempre irrilevante, ma utile conoscere il pattern.

```
1 <!--[if lte IE 8]>
2 <script>
3   document.createElement('header');
4   document.createElement('nav');
5   document.createElement('main');
6 </script>
7 <![endif]>
```


Checklist rapida

Prima di fare merge

- La pagina ha 1 solo `<main>`?
- I blocchi di navigazione principali sono in `<nav>` (non tutti i link)?
- Contenuti “pubblicabili” sono `<article>`?
- Sezioni tematiche sono `<section>` con heading sensato?
- Contenuti complementari sono in `<aside>`?
- Nav multiple hanno aria-label?
- Niente ARIA ridondante o in conflitto con HTML?

Anti-pattern comuni

Cose che vediamo spesso nei code review

- Usare `<section>` ovunque al posto di `<div>`.
- Molte `<nav>` non etichettate (screen reader: “navigation, navigation, ...”).
- Mettere contenuti principali dentro `<aside>` (o viceversa).
- Heading saltati “per estetica” (`h1` → `h4`) senza gerarchia.
- Aggiungere `role="main"` su `<main>` (ridondante) o ruoli incoerenti.

Mini-lab (10 minuti)

Refactoring guidato

- Parti da una pagina con `<div class="header/main/sidebar/footer">`.
- Sostituisci con tag semantici mantenendo le classi CSS esistenti.
- Aggiungi `<main id="main">` + skip link.
- Se ci sono più nav, aggiungi aria-label.
- Verifica: struttura di heading e landmark.

Esercizi

Set di esercizi (con starter e soluzioni)

- E1 — Refactor “div soup” → landmark HTML5
- E2 — Blog feed con <article>, <time>, <aside>
- E3 — Landmarks + ARIA (nav multiple, aria-current, skip link)
- E4 — Validazione: checklist + audit (Lighthouse/axe) e rubric

Nel pacchetto: README, file starter, file soluzione, e criteri di valutazione.